📚 Complete Learning Book: EV Routing System with AI

> **A Beginner-Friendly Guide to Understanding Electric Vehicle Route Optimization using Machine Learning**

📖 Table of Contents

🌟 1. Introduction

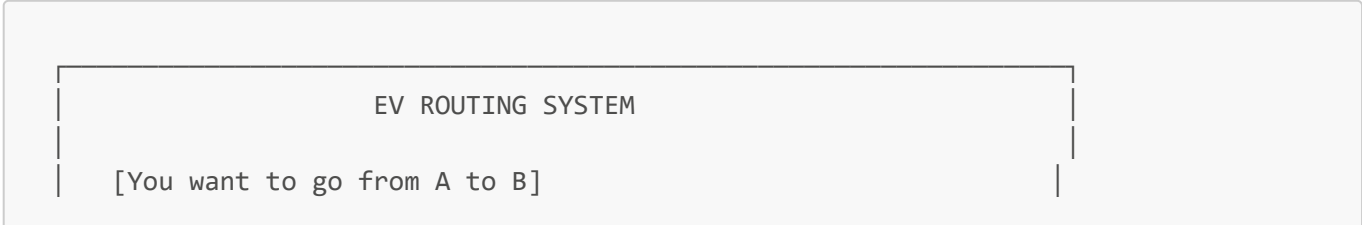
What is This Project About?

Imagine you have an electric car (EV) and need to drive from your home to a destination 50 km away. Unlike a gas car where you just drive, with an EV you need to consider:

1. **Battery Level**: Do I have enough charge to reach my destination?
2. **Traffic Conditions**: Will heavy traffic drain my battery faster?
3. **Charging Stations**: Where can I charge if needed?
4. **Route Efficiency**: Which path uses the least energy?

This project builds an **AI-powered system** that solves all these problems automatically!

The Big Picture



```
│              ↓                                                  │
│   [AI analyzes: traffic, battery, roads, charging stations]     │
│              ↓                                                  │
│   [AI suggests: "Take Route X, charge at Station Y"]            │
│              ↓                                                  │
│   [You arrive safely with optimal energy usage!]                │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Technologies Used

Technology	What It Does	Real-Life Analogy
Graph Theory	Represents roads as connections	City map with intersections
Q-Learning	Learns best decisions	Learning to drive through trial and error
GANs	Generates realistic traffic patterns	Creating fake but realistic traffic data
GNNs	Understands road network structure	Understanding how roads connect

🎓 2. Core Concepts & Prerequisites

Before diving deep, let's understand some fundamental concepts. Don't worry if these seem complex - we'll explain each one step by step!

Mathematics You'll Need

1. Basic Algebra

Variables, equations, and functions:

```
y = mx + b  (a straight line)
f(x) = x²   (a function)
```

2. Matrices and Vectors

A **vector** is a list of numbers:

```
v = [3, 5, 2]  ← This could represent: [battery%, distance, time]
```

A **matrix** is a table of numbers:

```
    A  B  C
A [ 0  1  0 ]   ← This shows: A connects to B
```

```
B [ 1  0  1 ]              B connects to A and C
C [ 0  1  0 ]              C connects to B
```

Real-life example: A contact list where 1 means "friends" and 0 means "not friends":

```
         Alice  Bob  Carol
Alice  [   -     1     1   ]   Alice is friends with Bob and Carol
Bob    [   1     -     0   ]   Bob is friends with Alice only
Carol  [   1     0     -   ]   Carol is friends with Alice only
```

3. Probability

The chance of something happening (0 to 1):

- 0 = never happens
- 1 = always happens
- 0.5 = 50% chance

Example: "There's a 70% chance of traffic jam at 5 PM" → P(jam|5PM) = 0.7

4. Basic Calculus (Conceptually)

Derivative: How fast something changes

- If you drive faster, fuel consumption increases
- The derivative tells us: "How much more fuel per extra km/h?"

Gradient: Direction of steepest increase

- Imagine standing on a hill - the gradient points uphill
- In AI, we use gradients to find the best solution

📊 Chapter 1: Graph Theory & Road Networks

What is a Graph?

A **graph** is NOT a chart or diagram - it's a mathematical structure!

```
Graph = Nodes (points) + Edges (connections)
```

Real-Life Example: Your City

```
    [Home]————————[Mall]
       |             |
```

```
        |           |
    [Park]─────[Work]
        |           |
        |           |
   [School]────[Hospital]
```

- **Nodes** (vertices): Home, Mall, Park, Work, School, Hospital
- **Edges**: Roads connecting them

Types of Graphs

1. Directed vs Undirected

Undirected: Roads go both ways

```
A ↔ B   (You can go A to B and B to A)
```

Directed: One-way streets

```
A → B    (You can only go A to B)
```

2. Weighted Graphs

Each edge has a "cost" (distance, time, energy):

```
      5 km
A ──────────→ B

      3 km      2 km
A ──────────→ C ──────────→ B
```

Going A→C→B is shorter (5 km) than A→B (5 km)... wait, they're equal! But what if:

- A→B has heavy traffic (30 min)
- A→C→B has light traffic (15 min total)

This is why we use **multiple weights**!

In This Project: Road Graph

```
# From road_graph.py - simplified explanation

class RoadGraph:
    """
```

```
    Our road network has:
    - Nodes: Intersections (grid_size × grid_size)
    - Edges: Roads with distance, time, energy cost
    - Special nodes: Charging stations
    """
```

How We Build the Road Network

```
Step 1: Create a grid of intersections

     0 ─── 1 ─── 2 ─── 3
     │     │     │     │
     4 ─── 5 ─── 6 ─── 7
     │     │     │     │
     8 ─── 9 ──10 ───11
     │     │     │     │
    12 ───13 ───14 ───15

Step 2: Add diagonal shortcuts (randomly)

     0 ─── 1 ─── 2 ─── 3
     │ \   │     │ \   │
     4 ─── 5 ─── 6 ─── 7
     │     │ \   │     │
     8 ─── 9 ──10 ───11
     │ \   │     │     │
    12 ───13 ───14 ───15

Step 3: Assign road types
        - Highway (fast, efficient)
        - Arterial (medium)
        - Residential (slow, stop-and-go)

Step 4: Place charging stations
        ⚡0 ─── 1 ─── 2 ─── ⚡3
         │     │     │     │
         4 ─── 5 ─── 6 ─── 7
         │     │     │     │
         8 ─── 9 ──10 ───11
         │     │     │     │
        ⚡12 ──13 ───14 ─── ⚡15
```

Edge Properties (What Each Road Knows)

Property	Description	Example
`distance_km`	Length of road	1.5 km
`base_time_minutes`	Time without traffic	3 min

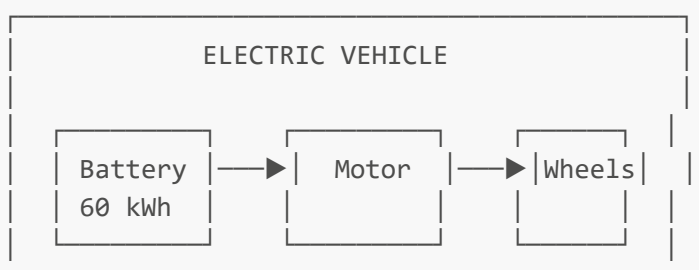
Property	Description	Example
road_type	Highway/Arterial/Residential	Highway
speed_limit_kmh	Maximum speed	80 km/h
base_energy_kwh_per_km	Energy consumption	0.18 kWh/km

The Code Explained

```
# Creating a road between two intersections
def _add_road(self, from_node, to_node):
    # Calculate distance using Pythagorean theorem
    # d = √[(x₂-x₁)² + (y₂-y₁)²]

    from_pos = self.graph.nodes[from_node]['pos']  # (x1, y1)
    to_pos = self.graph.nodes[to_node]['pos']      # (x2, y2)

    dx = to_pos[0] - from_pos[0]  # Δx = x2 - x1
    dy = to_pos[1] - from_pos[1]  # Δy = y2 - y1

    distance = √(dx² + dy²)  # Distance formula

    # Time = Distance / Speed
    # If speed = 50 km/h, and distance = 1 km
    # Time = 1/50 hours = 0.02 hours = 1.2 minutes
    base_time = (distance / 50.0) * 60  # Convert to minutes

    # Add the road with all properties
    self.graph.add_edge(from_node, to_node,
        distance_km=distance,
        base_time_minutes=base_time,
        road_type='arterial',
        speed_limit_kmh=50,
        base_energy_kwh_per_km=0.2
    )
```

Chapter 2: Electric Vehicle Fundamentals

How EVs Work (Simplified)



```
│                                                  │
│  Energy flows: Battery → Motor → Motion          │
└──────────────────────────────────────────────────┘
```

Key EV Concepts

1. Battery State of Charge (SoC)

What it is: Percentage of battery remaining (like your phone battery)

```
SoC = (Current Energy / Maximum Energy) × 100%

Example:
- Battery capacity: 60 kWh (kilowatt-hours)
- Current energy: 45 kWh
- SoC = (45/60) × 100% = 75%
```

Real-life analogy: Like a water tank

```
Full tank (100%) → ▓▓▓▓▓▓▓▓▓▓ ← 60 kWh
Current (75%)    → ▓▓▓▓▓▓▓▒▒▒ ← 45 kWh
Empty (0%)       → ▒▒▒▒▒▒▒▒▒▒ ← 0 kWh
```

2. Energy Consumption Rate

What it is: How much energy the EV uses per kilometer

```
Energy Rate = kWh / km

Typical values:
- Highway (constant speed): 0.15 - 0.18 kWh/km
- City (stop-and-go): 0.20 - 0.25 kWh/km
- Traffic jam: 0.25 - 0.35 kWh/km
```

Why traffic matters:

```
Constant Speed:
  ═══════════════════════▶  Energy: ▓▓▓▓▒▒▒▒

Stop-and-Go:
  ═▶ ▪ ═▶ ▪ ═▶ ▪ ═▶ ▪ ▶  Energy: ▓▓▓▓▓▓▓▒

Each stop and start wastes energy!
```

3. Range Estimation

Formula: $$\text{Range (km)} = \frac{\text{Remaining Energy (kWh)}}{\text{Consumption Rate (kWh/km)}}$$

Example:

```
Battery remaining: 30 kWh
Consumption rate: 0.2 kWh/km
Range = 30 / 0.2 = 150 km
```

4. Charging Time

Formula: $$\text{Charging Time (hours)} = \frac{\text{Energy Needed (kWh)}}{\text{Charger Power (kW)}}$$

Example:

```
Need to charge: 30 kWh
Charger power: 50 kW (fast charger)
Time = 30 / 50 = 0.6 hours = 36 minutes
```

In This Project: EVState Class

```python
@dataclass
class EVState:
    """
    Tracks everything about the EV's current condition
    """
    battery_soc: float = 100.0      # Battery % (0-100)
    battery_capacity_kwh: float = 60.0  # Max battery size
    energy_rate: float = 0.2        # kWh per km
    current_node: int = 0           # Where am I?
    time_minutes: int = 480         # What time? (8:00 AM)
    total_distance_km: float = 0.0  # How far traveled
    total_energy_kwh: float = 0.0   # How much energy used

    # Calculated properties
    @property
    def remaining_energy_kwh(self):
        """How much energy is left in the battery"""
        return (self.battery_soc / 100.0) * self.battery_capacity_kwh
        # If SoC=75%, Capacity=60: (75/100) × 60 = 45 kWh

    @property
    def estimated_range_km(self):
        """How far can we go?"""
        return self.remaining_energy_kwh / self.energy_rate
        # If remaining=45kWh, rate=0.2: 45/0.2 = 225 km
```

Energy Calculation Example

Let's trace a journey:

```
Start: Node 0, Battery 100% (60 kWh), Time 8:00 AM

Move 1: Node 0 → Node 1 (2 km highway)
  - Base energy: 2 km × 0.18 kWh/km = 0.36 kWh
  - Traffic (light): × 1.1 = 0.40 kWh
  - Battery: 60 - 0.40 = 59.60 kWh (99.3%)

Move 2: Node 1 → Node 5 (3 km residential)
  - Base energy: 3 km × 0.25 kWh/km = 0.75 kWh
  - Traffic (heavy): × 1.5 = 1.125 kWh
  - Battery: 59.60 - 1.125 = 58.475 kWh (97.5%)

And so on...
```

🗺️ Chapter 3: Pathfinding Algorithms

The Problem

Given a road network, how do we find the "best" path from A to B?

"Best" could mean:

- **Shortest** distance
- **Fastest** time
- **Most energy-efficient**
- **Cheapest** (fuel/energy cost)

Algorithm 1: Dijkstra's Algorithm

The most famous pathfinding algorithm! Finds the shortest path from one node to ALL other nodes.

The Concept

Imagine you're an ink drop on a map. You spread outward equally in all directions. The first time you reach any location is the shortest path!

```
Step 0: Start at A
        A[0]——B[∞]
        |      |
        C[∞]——D[∞]

Step 1: A spreads (distance 1)
        A[0]——B[1]
```

```
        |       |
        C[1]——D[∞]

Step 2: B and C spread (distance 2)
        A[0]——B[1]
        |       |
        C[1]——D[2]

Shortest path to D = 2
```

The Formula

For each unvisited node, we calculate: $$\text{new_distance} = \text{current_distance} + \text{edge_weight}$$

If new_distance < known_distance: update it!

Pseudocode

```
function Dijkstra(graph, start):
    distances = {all nodes: ∞}
    distances[start] = 0
    unvisited = all nodes

    while unvisited is not empty:
        current = node with smallest distance in unvisited

        for each neighbor of current:
            new_dist = distances[current] + edge_weight(current, neighbor)
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist

        remove current from unvisited

    return distances
```

In This Project

```python
# From road_graph.py
def shortest_path(self, source, destination):
    """Find shortest path by distance"""
    import networkx as nx
    return nx.dijkstra_path(self.graph, source, destination,
                            weight='distance_km')
```

Algorithm 2: A* (A-Star)

Dijkstra is great, but it explores in ALL directions. A* is smarter - it uses a **heuristic** to guide toward the goal.

The Concept

```
Dijkstra explores everywhere:        A* focuses toward goal:
         o o o o                              o
        o o A o o                           o A o
         o o o o                             o o o G
          o o o                                 o
```

The Formula

$$f(n) = g(n) + h(n)$$

Where:

- $f(n)$ = total estimated cost through node n
- $g(n)$ = actual cost from start to n
- $h(n)$ = **heuristic** estimate from n to goal

What's a Heuristic?

An educated guess! Common heuristics:

Euclidean Distance (straight line): $$h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$$

Manhattan Distance (grid-based): $$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

Example:

```
Current position: (2, 3)
Goal position: (5, 7)

Euclidean: √[(5-2)² + (7-3)²] = √[9 + 16] = √25 = 5

Manhattan: |5-2| + |7-3| = 3 + 4 = 7
```

Algorithm 3: Energy-Optimal Path

In this project, we don't just want the shortest path - we want the most **energy-efficient** one!

The Cost Function

```python
def calculate_edge_cost(self, edge, ev_state):
    """
    Calculate the REAL cost of traveling this edge
    """
    edge_data = self.graph.edges[edge]
    traffic = self.get_traffic_multiplier(edge, ev_state.time_minutes)
```

```
    distance = edge_data['distance_km']
    base_energy = edge_data['base_energy_kwh_per_km']

    # Traffic increases energy consumption!
    # Heavy traffic (1.5x) means 30% more energy
    energy_multiplier = 1 + (traffic - 1) * 0.3

    # Road type matters too
    if road_type == 'highway':
        energy_multiplier *= 0.9  # 10% more efficient
    elif road_type == 'residential':
        energy_multiplier *= 1.2  # 20% less efficient

    total_energy = distance * base_energy * energy_multiplier

    return total_energy
```

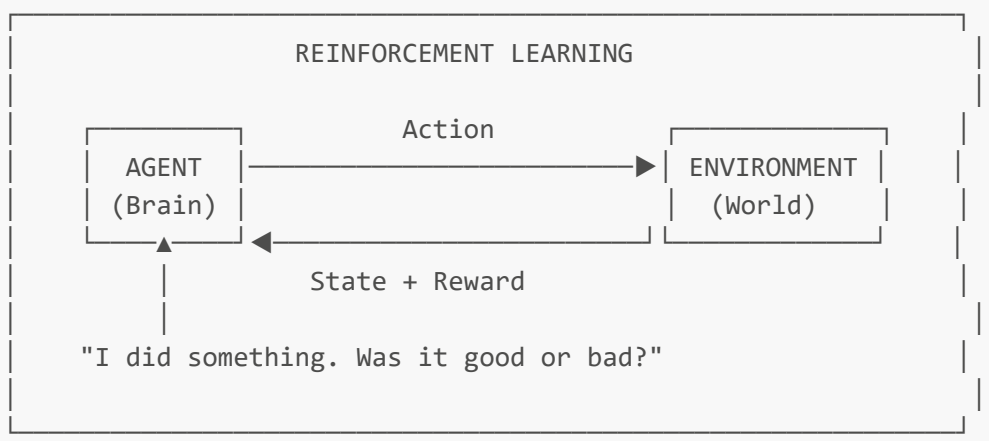
Example: Choosing Between Routes

```
Route A: Highway, 10 km, light traffic
  Energy = 10 × 0.18 × 0.9 × 1.0 = 1.62 kWh

Route B: Residential, 8 km, heavy traffic
  Energy = 8 × 0.25 × 1.2 × 1.5 = 3.60 kWh

Winner: Route A (even though it's longer!)
```

🎮 Chapter 4: Reinforcement Learning & Q-Learning

What is Reinforcement Learning?

RL is how machines learn through **trial and error**, just like humans!



Real-Life Example: Learning to Drive

```
You (Agent) → Drive to work (Environment)

Day 1: Take route A → Get stuck in traffic (Negative reward: -10)
Day 2: Take route B → Arrive quickly (Positive reward: +20)
Day 3: Take route B again → Still good! (+20)
Day 4: Take route C → Find shortcut! (+30)

Over time, you LEARN which routes are best!
```

Key RL Concepts

1. State (S)

Where you are / what you observe

```
EV State Example:
- Current position: Node 5
- Battery: 60%
- Time: 8:30 AM
- Traffic level: Heavy

State = (5, 60, 8.5, heavy)
```

2. Action (A)

What you can do

```
Possible Actions:
- Action 0: Go North
- Action 1: Go East
- Action 2: Go South
- Action 3: Go West
- Action 4: Charge at station
```

3. Reward (R)

Feedback on your action

```
Rewards in this project:
- Reach destination: +100
- Move closer to destination: +5
- Use energy: -1 per kWh
```

```
- Run out of battery: -100
- Each step (time penalty): -0.1
```

4. Policy (π)

Your strategy for choosing actions

```
Policy examples:
- Random: Always pick randomly
- Greedy: Always pick best known action
- ε-greedy: Usually best, sometimes random
```

Q-Learning Explained

Q-Learning is a specific RL algorithm that learns a **Q-Table** - a lookup table of "how good is each action in each state?"

The Q-Table

```
                    Actions
                Go_N   Go_E   Go_S   Go_W   Charge
          ┌──────────────────────────────────────────┐
State 1   │  2.5    3.1    1.2    0.8    0.0        │
State 2   │  1.8    2.4    4.2    1.1    5.5        │
State 3   │  0.5    0.9    1.5    2.3    1.0        │
  ...     │  ...    ...    ...    ...    ...        │
          └──────────────────────────────────────────┘

Q(state, action) = Expected future reward

In State 2: Charging (5.5) is the best action!
```

The Q-Learning Formula

This is the **Bellman Equation** - the heart of Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

Let's break this down:

Symbol	Meaning	Analogy
$Q(s,a)$	Current Q-value	"How good is action a in state s?"
α	Learning rate (0-1)	"How much do I trust new info?"
r	Immediate reward	"What did I get right now?"

Symbol	Meaning	Analogy
γ	Discount factor (0-1)	"How much do I care about future?"
$\max_{a'} Q(s',a')$	Best future value	"What's the best I can do from here?"

Step-by-Step Example

```
Current situation:
- State s: At node 5, battery 80%
- Action a: Move East (to node 6)
- Reward r: +5 (moved closer to destination)
- New state s': At node 6, battery 75%
- Learning rate α: 0.1
- Discount factor γ: 0.95

Current Q-value: Q(s, "East") = 2.0
Best future Q-value: max Q(s', all_actions) = 8.0

Update:
Q(s, "East") = 2.0 + 0.1 × [5 + 0.95 × 8.0 - 2.0]
             = 2.0 + 0.1 × [5 + 7.6 - 2.0]
             = 2.0 + 0.1 × 10.6
             = 2.0 + 1.06
             = 3.06

New Q-value: 3.06 (increased because this was a good move!)
```

Exploration vs Exploitation

A key challenge: should we try new things or stick with what works?

Exploration: Try random actions to discover new strategies **Exploitation**: Use the best known action

ε-greedy Strategy:

```
ε = 0.1 (10% exploration)

if random() < ε:
    action = random_action()    # Explore (10% of time)
else:
    action = best_known_action()  # Exploit (90% of time)
```

Decay: Start with high exploration, gradually reduce:

```
Episode 1:   ε = 1.0    (100% random - know nothing)
Episode 100: ε = 0.5    (50% random - learning)
```

```
Episode 500: ε = 0.1    (10% random - mostly learned)
Episode 1000: ε = 0.01  (1% random - almost expert)
```

In This Project: QLearningAgent

```python
class QLearningAgent:
    def __init__(self,
                 action_space=5,           # 4 directions + charge
                 learning_rate=0.1,        # α
                 discount_factor=0.95,     # γ
                 exploration_rate=1.0,     # ε (starts high)
                 exploration_decay=0.995): # ε decreases each episode

        self.q_table = {}  # Empty at first!

    def choose_action(self, state, training=True):
        """ε-greedy action selection"""
        if training and random() < self.exploration_rate:
            return random_action()  # Explore
        else:
            return argmax(self.q_table[state])  # Exploit

    def learn(self, state, action, reward, next_state, done):
        """Update Q-table using Bellman equation"""
        current_q = self.q_table[state][action]

        if done:
            max_next_q = 0  # No future after terminal state
        else:
            max_next_q = max(self.q_table[next_state])

        # THE BELLMAN UPDATE:
        new_q = current_q + self.learning_rate * (
            reward + self.discount_factor * max_next_q - current_q
        )

        self.q_table[state][action] = new_q
```

🧠 Chapter 5: Neural Networks Basics

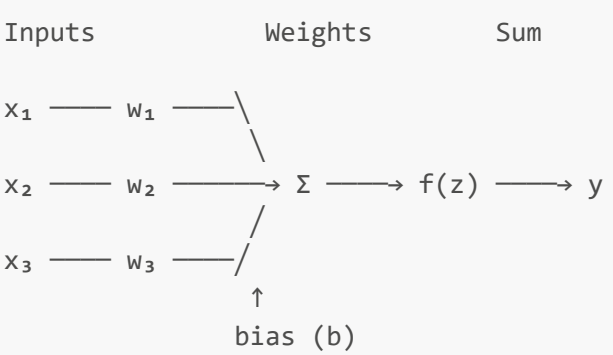
What is a Neural Network?

A neural network is a mathematical model inspired by the human brain.

```
Human Brain:                    Artificial Neural Network:

   (Neurons)                       (Nodes)
```

```
86 billion neurons              100s to millions of nodes
Learn from experience           Learn from data
```

The Basic Building Block: A Neuron



The Math

Step 1: Weighted Sum $$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

Or in vector form: $$z = \mathbf{w} \cdot \mathbf{x} + b$$

Step 2: Activation Function $$y = f(z)$$

Common activation functions:

Function	Formula	Range	Use
Sigmoid	$\frac{1}{1+e^{-z}}$	(0, 1)	Probability
ReLU	$\max(0, z)$	[0, ∞)	Hidden layers
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	(-1, 1)	Hidden layers

Example: Is This a Good Route?

```
Inputs:
  x₁ = distance = 10 km (normalized: 0.5)
  x₂ = energy = 3 kWh (normalized: 0.3)
  x₃ = traffic = heavy (normalized: 0.8)

Weights (learned):
  w₁ = -0.5 (distance is bad)
  w₂ = -0.8 (energy use is bad)
```

```
  w₃ = -0.6 (traffic is bad)
  b = 1.0 (base "goodness")

Calculate:
  z = (-0.5)(0.5) + (-0.8)(0.3) + (-0.6)(0.8) + 1.0
    = -0.25 - 0.24 - 0.48 + 1.0
    = 0.03

Apply sigmoid:
  y = 1/(1 + e^(-0.03)) = 0.507

Output: 50.7% chance this is a good route
```

Layers of Neurons

Real neural networks have multiple layers:

```
Input Layer      Hidden Layer(s)     Output Layer
(Features)       (Processing)        (Prediction)

    o               o o                 o
    o               o o                 o
    o        →      o o          →      o
    o               o o
    o               o o

 5 inputs         2 × 5 = 10          3 outputs
                   neurons
```

Why hidden layers?

- Input layer: raw data (can't learn)
- Hidden layers: learn patterns and features
- Output layer: final prediction

More layers = more complex patterns (but harder to train)

Training: How Networks Learn

1. Forward Pass

Push data through network, get prediction

2. Calculate Loss

Compare prediction to truth

$$\text{Loss} = \frac{1}{2}(y_{predicted} - y_{actual})^2$$

3. Backward Pass (Backpropagation)

Calculate how each weight contributed to the error

Using chain rule: $$\frac{\partial Loss}{\partial w} = \frac{\partial Loss}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

4. Update Weights

Adjust weights to reduce loss

$$w_{new} = w_{old} - \text{learning_rate} \times \frac{\partial Loss}{\partial w}$$

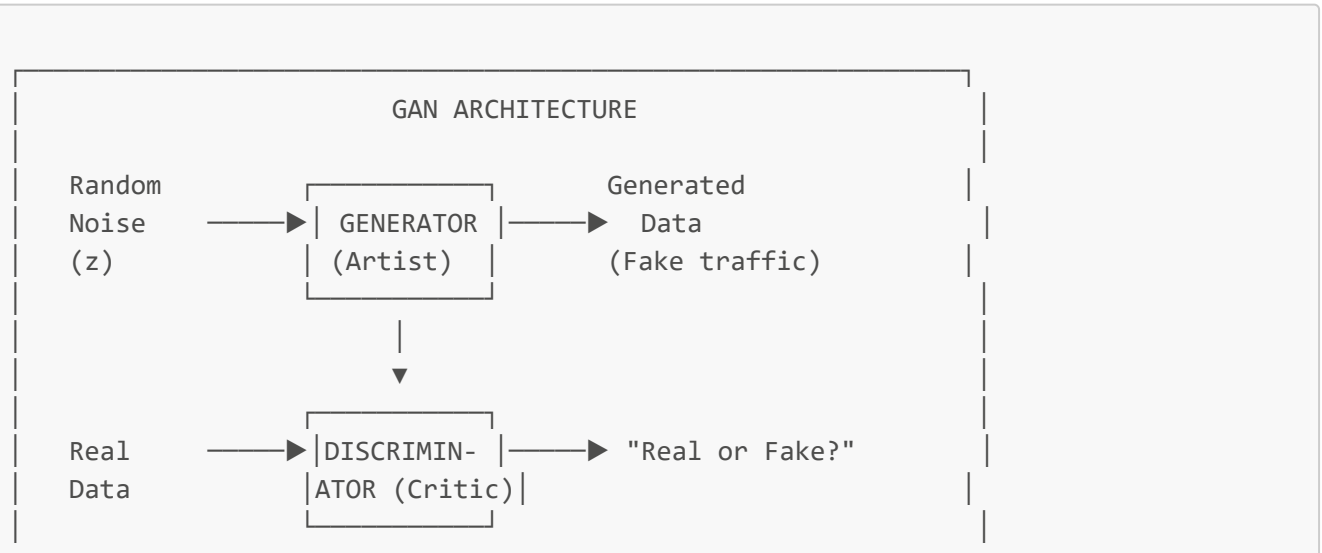
Visual Example

```
Forward Pass:
Input [0.5, 0.3] → Hidden [0.7, 0.4] → Output [0.6]
                                            ↓
Truth: [0.9]                        Loss = (0.6-0.9)² = 0.09

Backward Pass:
Loss 0.09 → How much did each weight contribute?
          → Adjust weights to reduce loss

Next Forward Pass:
Input [0.5, 0.3] → Hidden [0.72, 0.38] → Output [0.72]
                                              ↓
                                     Loss = (0.72-0.9)² = 0.03

Loss decreased! Network is learning!
```

🎭 Chapter 6: Generative Adversarial Networks (GANs)

What is a GAN?

A GAN is two neural networks competing against each other:



```
│                                                                 │
│   Generator: "I'll make fakes so good you can't tell!"          │
│   Discriminator: "I'll always catch the fakes!"                 │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Real-Life Analogy: Art Forger vs Detective

```
Art Forger (Generator):
  - Tries to create fake paintings
  - Gets better at fooling the detective
  - Goal: Make undetectable fakes

Detective (Discriminator):
  - Examines paintings
  - Decides: real or fake?
  - Goal: Never be fooled

Result: Both get REALLY good at their jobs!
The forger eventually makes perfect copies.
```

The Mathematics

Generator (G)

Takes random noise, produces fake data: $$G(z) \rightarrow \text{fake data}$$

Discriminator (D)

Takes data, outputs probability of being real: $$D(x) \rightarrow P(\text{x is real})$$

The Loss Functions

Discriminator wants to:

- Say "Real!" for real data: $D(x) \rightarrow 1$
- Say "Fake!" for generated data: $D(G(z)) \rightarrow 0$

Generator wants to:

- Fool discriminator: $D(G(z)) \rightarrow 1$

The Minimax Game

$$\min_G \max_D \mathbb{E}*{x \sim \text{real}}[\log D(x)] + \mathbb{E}*{z \sim \text{noise}}[\log(1 - D(G(z)))]$$

In simpler terms:

- D maximizes: "Correctly identify real AND fake"
- G minimizes: "Make D think fake is real"

Training Process

```
Round 1:
  Generator: Creates random noise patterns
  Discriminator: "Easy! That's obviously fake!" (95% confident)

Round 100:
  Generator: Creates patterns with some structure
  Discriminator: "Still fake, but better" (80% confident)

Round 1000:
  Generator: Creates realistic traffic patterns
  Discriminator: "Hmm... maybe real?" (60% confident)

Round 10000:
  Generator: Creates near-perfect traffic patterns
  Discriminator: "I honestly can't tell!" (50% confident)

Training complete! Generator can create realistic data!
```

In This Project: SG-GAN for Traffic

What We Generate

Realistic 24-hour traffic patterns for all roads:

```
Generated Traffic (20 roads × 24 hours):

        Hour: 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
23
Road 1:     [0.2 0.1 0.1 0.1 0.2 0.3 0.5 1.3 1.6 1.2 0.8 0.9 1.1 0.9 0.8 0.9 1.1
1.5 1.7 1.3 0.9 0.6 0.4 0.3]
Road 2:     [0.3 0.2 0.1 0.2 0.2 0.4 0.6 1.2 1.5 1.3 0.9 1.0 1.0 0.8 0.9 1.0 1.2
1.6 1.8 1.4 0.8 0.5 0.3 0.2]
...

Notice the patterns:
- Low at night (0.1-0.3)
- Morning peak (1.3-1.6) around hours 7-8
- Evening peak (1.5-1.8) around hours 17-18
```

The Generator Architecture

```python
class SGGANGenerator(Model):
    """
    Input:
      - Noise vector (100 random numbers)
```

```
        - EV state (5 numbers: battery, energy, range, time, rate)
        - Conditions (10 numbers: source/dest position, etc.)

    Output:
        - Traffic pattern (20 roads × 24 hours)
    """

    def __init__(self):
        # Process noise into initial features
        self.dense1 = Dense(256, activation='relu')

        # Build up complexity
        self.dense2 = Dense(512, activation='relu')
        self.dense3 = Dense(1024, activation='relu')

        # Final output: 20 × 24 = 480 values
        self.dense4 = Dense(480)
        self.reshape = Reshape((20, 24))

    def call(self, inputs):
        x = concatenate([noise, ev_state, conditions])
        x = self.dense1(x)
        x = self.dense2(x)
        x = self.dense3(x)
        x = self.dense4(x)
        x = tanh(x)  # Output in range [-1, 1]
        x = self.reshape(x)  # Shape: (20, 24)
        return x
```

The Discriminator Architecture

```
class SGGANDiscriminator(Model):
    """
    Input:
        - Traffic pattern (20 roads × 24 hours)
        - EV state (5 numbers)
        - Conditions (10 numbers)

    Output (Multi-task):
        - validity: Is this real traffic? (0-1)
        - energy_feasibility: Is this energy-feasible? (0-1)
        - realism: Does this look realistic? (0-1)
    """

    def __init__(self):
        self.flatten = Flatten()  # 20×24 → 480
        self.dense1 = Dense(512, activation='leaky_relu')
        self.dense2 = Dense(256, activation='leaky_relu')
        self.dense3 = Dense(128, activation='leaky_relu')

        # Multiple output heads for different tasks
```

```
        self.validity_head = Dense(1, activation='sigmoid')
        self.energy_head = Dense(1, activation='sigmoid')
        self.realism_head = Dense(1, activation='sigmoid')
```

🕸️ Chapter 7: Graph Neural Networks (GNNs)

Why Do We Need GNNs?

Regular neural networks work with fixed-size inputs:

- Images: 224×224 pixels
- Text: Sequence of words
- Tables: Fixed columns

But road networks are **graphs** with:

- Variable number of nodes
- Variable connections
- Complex structure

```
Regular NN input:        Graph input:
[x₁, x₂, x₃, x₄]        o──o
                         \ /
Fixed size = 4            o──o──o
                              \
                               o

                      Variable size, complex connections
```

The Core Idea: Message Passing

GNNs work by passing information between connected nodes:

```
Before message passing:        After message passing:

    o(3)                           o(4.5)
    |                              |
    o(2)──o(5)                     o(3.3)──o(5.2)
    |                              |
    o(1)                           o(2.5)

Each node now knows about its neighbors!
```

Graph Convolutional Layer (GCN)

The Formula

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \frac{1}{c_{vu}} W^{(l)} h_u^{(l)} \right)$$

Let's break this down:

- $h_v^{(l)}$ = features of node v at layer l
- $\mathcal{N}(v)$ = neighbors of node v
- $W^{(l)}$ = learnable weight matrix
- c_{vu} = normalization factor
- σ = activation function (like ReLU)

In Plain English

```
New node features = Activation(
    Sum of (neighbor features × weights) / normalization
)

Example for node B:
    A(3)
     │
B(2)─┼─C(5)
     │
    D(1)

B's neighbors: A, C, D
B's new features = ReLU(Average([3, 5, 1]) × W + B's own features)
                 = ReLU(3 × W + 2)
```

The Code

```
class GraphConvLayer(layers.Layer):
    """
    Graph Convolutional Layer
    """

    def call(self, node_features, adjacency):
        """
        node_features: (batch, nodes, features)
        adjacency: (batch, nodes, nodes) - who connects to who
        """

        # Add self-loops (each node connects to itself)
        identity = eye(num_nodes)
        adj_with_self = adjacency + identity

        # Normalize: D^(-1/2) × A × D^(-1/2)
        # This balances high-degree and low-degree nodes
        degree = sum(adj_with_self, axis=-1)  # How many connections
        norm = 1 / sqrt(degree)
```

```
        adj_normalized = adj_with_self * norm * norm.T

        # Aggregate neighbor features
        aggregated = adj_normalized @ node_features  # Matrix multiplication

        # Transform with learnable weights
        output = aggregated @ self.W + self.b

        # Apply activation
        return relu(output)
```

Graph Attention Layer (GAT)

GCN treats all neighbors equally. But some neighbors might be more important!

GAT uses attention: "Pay more attention to important neighbors"

The Attention Mechanism

```
Node B's neighbors: A, C, D

Without attention (GCN):
  B_new = (A + C + D) / 3  (equal weight)

With attention (GAT):
  Attention scores:
    α(B,A) = 0.5  (A is very important)
    α(B,C) = 0.3  (C is somewhat important)
    α(B,D) = 0.2  (D is less important)

  B_new = 0.5×A + 0.3×C + 0.2×D  (weighted average)
```

How Attention Scores Are Calculated

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(a^T [W h_v || W h_u]))}{\sum_{k \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(a^T [W h_v || W h_k]))}$$

In simpler terms:

1. Transform node features: $W h_v$, $W h_u$
2. Concatenate: $[W h_v || W h_u]$
3. Score with attention vector: $a^T [\cdot]$
4. Apply LeakyReLU and softmax

Multi-Head Attention

Use multiple attention "heads" to capture different relationship types:

```
Head 1: Focus on distance relationships
Head 2: Focus on traffic relationships
Head 3: Focus on energy relationships
Head 4: Focus on time relationships

Final = Concatenate(Head1, Head2, Head3, Head4)
```

In This Project: GNN Route GAN

We use GNNs in our route generator to understand the road network structure:

```
class GNNRouteGenerator(Model):
    """
    Uses GNN to generate routes that respect graph structure
    """

    def __init__(self, num_nodes=100):
        # Input processing
        self.noise_dense = Dense(128)
        self.ev_dense = Dense(128)

        # GNN layers to understand road network
        self.gnn_layers = [
            GraphConvLayer(128),  # Layer 1
            GraphConvLayer(128),  # Layer 2
            GraphConvLayer(128),  # Layer 3
        ]

        # Attention layer for important connections
        self.attention = GraphAttentionLayer(units=32, num_heads=4)

        # Output: probability for each node being in route
        self.output_dense = Dense(1, activation='sigmoid')

    def call(self, inputs):
        noise, ev_state, source_dest = inputs

        # Create initial node features
        node_features = self.create_node_features(noise, ev_state)

        # Process through GNN layers
        for gnn in self.gnn_layers:
            node_features = gnn(node_features, adjacency)

        # Apply attention
        node_features = self.attention(node_features, adjacency)

        # Get probability for each node
        node_probs = self.output_dense(node_features)
```

```
        return node_probs  # Shape: (batch, num_nodes, 1)
```

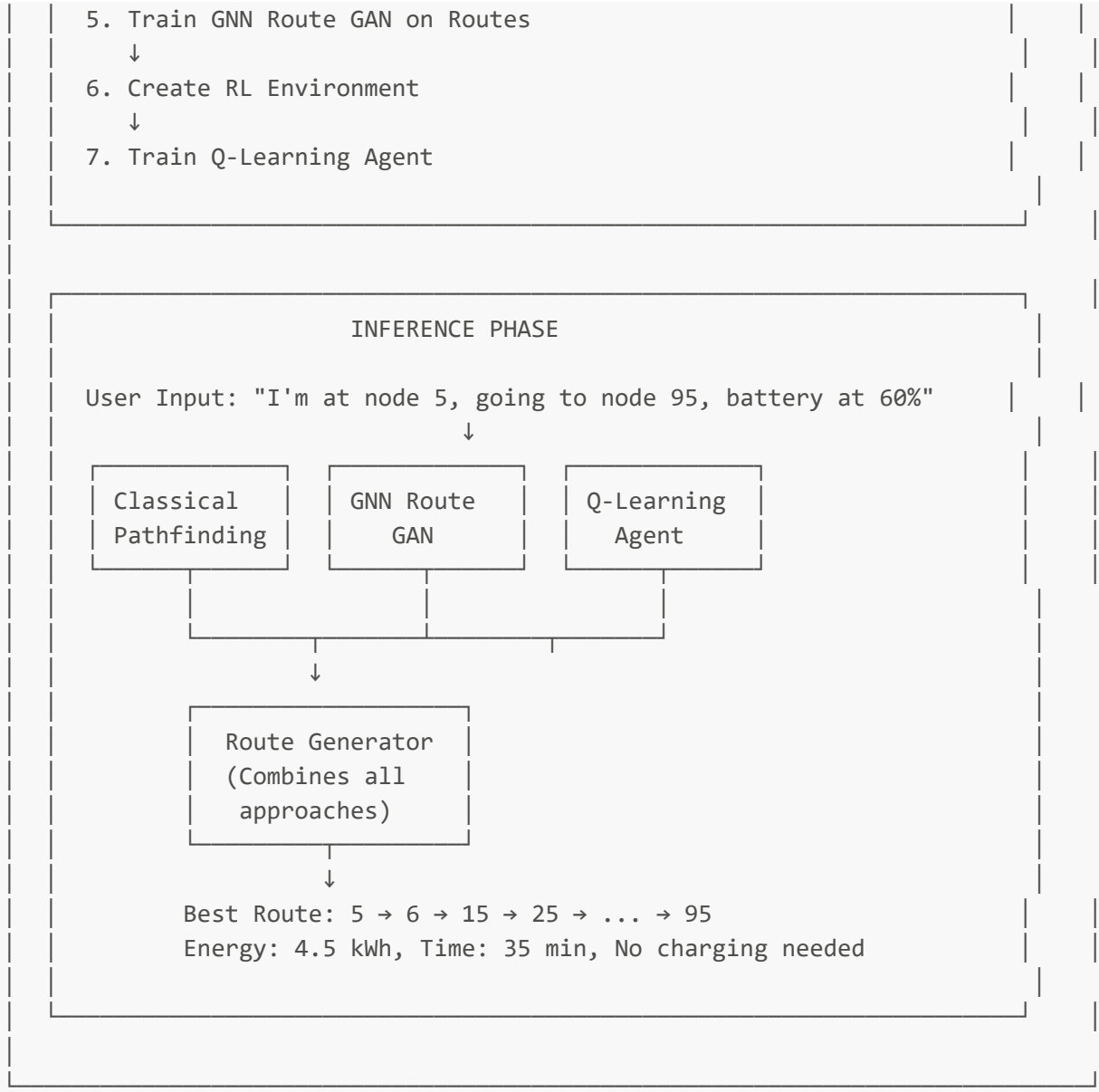
How Routes Are Generated

```
1. Input random noise + EV state + source/destination

2. GNN processes the road graph:
   Layer 1: Each node learns about immediate neighbors
   Layer 2: Each node learns about 2-hop neighbors
   Layer 3: Each node learns about 3-hop neighbors

3. Attention focuses on relevant connections

4. Output: Probability for each node being in the route

   Node 0 (source):    0.95  ← Very likely
   Node 1:             0.72  ← Probably in route
   Node 2:             0.15  ← Probably not
   Node 3:             0.68  ← Probably in route
   ...
   Node 99 (dest):     0.92  ← Very likely

5. Greedy decoding: Start at source, always move to
   highest-probability neighbor until reaching destination

   Route: 0 → 1 → 3 → 7 → 15 → ... → 99
```

🏗️ Chapter 8: The Complete System Architecture

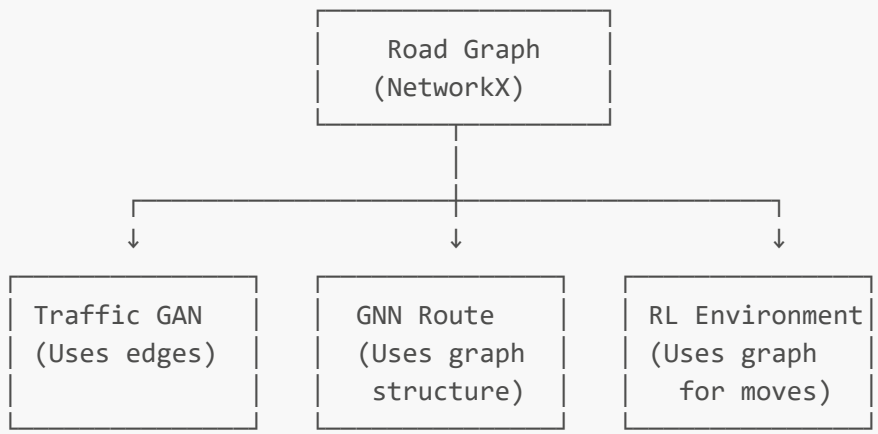
The Big Picture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        EV ROUTING SYSTEM                                 │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────┐       │
│   │                      TRAINING PHASE                          │       │
│   │                                                              │       │
│   │  1. Create Road Network (RoadGraph)                          │       │
│   │     ↓                                                        │       │
│   │  2. Generate Synthetic Traffic Data                          │       │
│   │     ↓                                                        │       │
│   │  3. Train SG-GAN on Traffic Patterns                         │       │
│   │     ↓                                                        │       │
│   │  4. Generate Historical Routes                               │       │
│   │     ↓                                                        │       │
```



Component Interactions

1. Road Graph ↔ Everything



2. Data Flow During Training

```
Step 1: Create Road Graph
        ↓
        100 nodes, 400+ edges, 8 charging stations

Step 2: Generate Traffic Data
        ↓
        create_synthetic_traffic(500 samples)
        → Shape: (500, 20, 24)

Step 3: Train SG-GAN
        ↓
        Input: Traffic samples
        Train: Generator vs Discriminator
        Output: Generator can create realistic traffic

Step 4: Generate Historical Routes
        ↓
        For random (source, destination) pairs:
          - Find valid paths
          - Record: path, energy, time
        Output: 300 training routes

Step 5: Train GNN Route GAN
        ↓
        Input: Historical routes as graph data
        Train: Generator creates routes
               Discriminator validates:
               - Route validity
               - Energy feasibility
               - Traffic realism
               - Graph connectivity

Step 6: Create RL Environment
        ↓
        State: (position, battery, time, traffic, ...)
        Actions: (north, east, south, west, charge)
        Rewards: (+100 destination, -100 empty battery, ...)

Step 7: Train Q-Learning Agent
        ↓
        For 500 episodes:
          - Reset environment
          - Agent takes actions
          - Update Q-table
          - Decay exploration
        Output: Trained Q-table
```

3. Data Flow During Inference

```
User Request:
  Source: Node 5
  Destination: Node 95
  Battery: 60%
  Time: 8:00 AM

        ↓

┌─────────────────────────────────────────────────────────┐
│                  Route Generator                        │
│                                                         │
│  Method 1: Shortest Path (Dijkstra)                     │
│    → Route A: 5→15→25→35→45→55→65→75→85→95              │
│    → Distance: 9 km, Energy: 2.1 kWh                    │
│                                                         │
│  Method 2: Energy-Optimal Path                          │
│    → Route B: 5→6→16→26→36→46→56→66→76→86→96→95         │
│    → Distance: 11 km, Energy: 1.8 kWh                   │
│                                                         │
│  Method 3: GNN-Guided Route                             │
│    → Route C: 5→15→26→36→47→57→67→77→87→95              │
│    → Distance: 10 km, Energy: 1.9 kWh                   │
│                                                         │
│  Method 4: K-Shortest Paths (3 alternatives)            │
│    → Routes D, E, F with varying trade-offs             │
│                                                         │
│  Method 5: Via Charging Station (if battery low)        │
│    → Route G: 5→...→Station→...→95                      │
│                                                         │
└─────────────────────────────────────────────────────────┘

        ↓

Scoring & Ranking:
  Route B: Score 0.95 (best energy)
  Route C: Score 0.91 (good balance)
  Route A: Score 0.85 (shortest but more energy)
  ...

        ↓

Output: "Take Route B. Estimated 35 minutes, 1.8 kWh energy use.
        No charging needed. You'll arrive with 57% battery."
```

The RL Environment in Detail

```python
class EVRoutingEnvironment(gym.Env):
    """
    Gymnasium-compatible RL Environment
    """
```

```
    # OBSERVATION SPACE (what the agent sees)
    # 8 normalized values between 0 and 1:
    observation_space = [
        node_x,           # X position (0-1)
        node_y,           # Y position (0-1)
        battery_soc,      # Battery % (0-1)
        dist_to_dest,     # Distance to goal (0-1)
        time_of_day,      # Current hour / 24 (0-1)
        traffic_level,    # Current traffic (0-1)
        is_at_charging,   # 1 if at station, 0 otherwise
        can_reach_dest    # 1 if enough battery, 0 otherwise
    ]

    # ACTION SPACE (what the agent can do)
    # Discrete(5): Actions 0-4
    actions = [
        0: "Move to neighbor 0",
        1: "Move to neighbor 1",
        2: "Move to neighbor 2",
        3: "Move to neighbor 3",
        4: "Charge at current station"
    ]

    # REWARD FUNCTION (feedback to agent)
    def calculate_reward(self):
        if reached_destination:
            return +100.0   # Big reward!
        elif battery_empty:
            return -100.0   # Big penalty!
        elif moved_closer:
            return +5.0 * progress   # Encourage progress
        else:
            return -0.1     # Small penalty for each step (time cost)

        # Also subtract energy used
        reward -= energy_used * 1.0
```

💻 Chapter 9: Code Walkthrough

File Structure Overview

```
src/
├── main.py              # Entry point - orchestrates everything
├── road_graph.py        # Road network + EV state
├── traffic_generator.py # SG-GAN for traffic patterns
├── gnn_route_generator.py # GNN-based route generation
├── environment.py       # RL environment
├── q_learning_agent.py  # Q-Learning implementation
```

```
├── route_generator.py   # Route planning algorithms
└── evaluate.py          # Model evaluation
```

File-by-File Explanation

1. main.py - The Orchestrator

```python
# main.py orchestrates the entire pipeline

class EVRoutingSystem:
    """Main system class that coordinates all components"""

    def __init__(self, config):
        # Initialize all components as None
        self.road_graph = None
        self.gan = None
        self.gnn_gan = None
        self.environment = None
        self.agent = None
        self.route_generator = None

    def run_training_pipeline(self):
        """Execute the 7-step training process"""

        # Step 1: Create road network
        self.step1_create_road_network()

        # Step 2: Generate traffic data
        traffic_data = self.step2_generate_traffic_data()

        # Step 3: Train SG-GAN
        self.step3_train_gan(traffic_data)

        # Step 3b: Train GNN Route GAN (optional)
        if self.config['use_gnn_gan']:
            self.step3b_train_gnn_gan()

        # Step 4: Create RL environment
        self.step4_create_environment()

        # Step 5: Train Q-Learning agent
        self.step5_train_agent()

        # Step 6: Create route generator
        self.step6_create_route_generator()

        # Step 7: Evaluate system
        self.step7_evaluate()
```

2. road_graph.py - The Foundation

```python
# road_graph.py - Creates and manages the road network

class RoadGraph:
    """
    Graph-based road network

    Key Responsibilities:
    1. Create grid of nodes (intersections)
    2. Connect with edges (roads)
    3. Store traffic patterns
    4. Calculate route costs
    5. Manage charging stations
    """

    def __init__(self, grid_size=10, seed=42):
        self.graph = networkx.DiGraph()
        self._build_grid_network()      # Create basic grid
        self._add_diagonal_roads()       # Add shortcuts
        self._assign_road_types()        # Highway/Arterial/Residential
        self._add_charging_stations()    # Place 8 stations
        self.traffic_patterns = self._generate_base_traffic_patterns()

    def shortest_path(self, source, destination):
        """Find shortest path by distance"""
        return nx.dijkstra_path(self.graph, source, destination,
                                weight='distance_km')

    def energy_optimal_path(self, source, destination, ev_state):
        """Find path that uses least energy"""
        def energy_weight(u, v, d):
            return self.calculate_edge_cost((u, v), ev_state)['energy_kwh']
        return nx.dijkstra_path(self.graph, source, destination,
                                weight=energy_weight)

    def calculate_edge_cost(self, edge, ev_state):
        """Calculate energy, time, distance for traversing an edge"""
        # Get base values
        distance = edge_data['distance_km']
        base_energy = edge_data['base_energy_kwh_per_km']

        # Adjust for traffic
        traffic = self.get_traffic_multiplier(edge, ev_state.time_minutes)
        energy_multiplier = 1 + (traffic - 1) * 0.3

        # Adjust for road type
        if road_type == 'highway':
            energy_multiplier *= 0.9
        elif road_type == 'residential':
            energy_multiplier *= 1.2

        return {
            'energy_kwh': distance * base_energy * energy_multiplier,
```

```
            'time_minutes': base_time * traffic,
            'distance_km': distance,
            'feasible': ev_state.remaining_energy >= energy_needed
        }


@dataclass
class EVState:
    """
    Tracks EV condition

    This is like the EV's "status screen":
    - Battery level
    - Current position
    - Trip statistics
    """
    battery_soc: float = 100.0        # Battery percentage
    battery_capacity_kwh: float = 60.0 # Battery size
    energy_rate: float = 0.2          # kWh per km
    current_node: int = 0             # Current position
    time_minutes: int = 480           # Time (8:00 AM)
    total_distance_km: float = 0.0    # Trip distance
    total_energy_kwh: float = 0.0     # Trip energy used
```

3. traffic_generator.py - SG-GAN

```
# traffic_generator.py - Generates realistic traffic patterns

class SGGANTrafficGenerator:
    """
    Complete GAN for traffic generation

    Architecture:
    - Generator: Noise → Traffic pattern
    - Discriminator: Traffic pattern → Real/Fake score
    """

    def __init__(self, input_shape=(20, 24), noise_dim=100):
        self.generator = SGGANGenerator(output_shape=input_shape)
        self.discriminator = SGGANDiscriminator(input_shape=input_shape)

        # Optimizers
        self.g_optimizer = Adam(learning_rate=0.0002, beta_1=0.5)
        self.d_optimizer = Adam(learning_rate=0.0002, beta_1=0.5)

    def train_step(self, real_traffic, ev_state, conditions):
        """Single training step"""

        # Generate noise
        noise = tf.random.normal([batch_size, self.noise_dim])
```

```
        # ===== TRAIN DISCRIMINATOR =====
        with tf.GradientTape() as d_tape:
            # Generate fake traffic
            fake_traffic = self.generator([noise, ev_state, conditions])

            # Discriminator predictions
            real_output = self.discriminator([real_traffic, ev_state, conditions])
            fake_output = self.discriminator([fake_traffic, ev_state, conditions])

            # Discriminator loss: real → 1, fake → 0
            d_loss_real = binary_crossentropy(ones, real_output['combined'])
            d_loss_fake = binary_crossentropy(zeros, fake_output['combined'])
            d_loss = (d_loss_real + d_loss_fake) / 2

        # Update discriminator
        d_gradients = d_tape.gradient(d_loss,
self.discriminator.trainable_variables)
        self.d_optimizer.apply_gradients(zip(d_gradients,
self.discriminator.trainable_variables))

        # ===== TRAIN GENERATOR =====
        with tf.GradientTape() as g_tape:
            # Generate fake traffic
            fake_traffic = self.generator([noise, ev_state, conditions])

            # Discriminator prediction
            fake_output = self.discriminator([fake_traffic, ev_state, conditions])

            # Generator loss: fake → 1 (fool discriminator)
            g_loss = binary_crossentropy(ones, fake_output['combined'])

        # Update generator
        g_gradients = g_tape.gradient(g_loss, self.generator.trainable_variables)
        self.g_optimizer.apply_gradients(zip(g_gradients,
self.generator.trainable_variables))

        return {'d_loss': d_loss, 'g_loss': g_loss}

    def generate_traffic_scenarios(self, n_samples):
        """Generate new traffic patterns"""
        noise = tf.random.normal([n_samples, self.noise_dim])
        ev_state = tf.zeros([n_samples, 5])
        conditions = tf.zeros([n_samples, 10])

        generated = self.generator([noise, ev_state, conditions], training=False)

        # Convert from [-1, 1] to [0, 1]
        return (generated.numpy() + 1) / 2
```

4. q_learning_agent.py - The Brain

```python
# q_learning_agent.py - Learns optimal routing decisions

class QLearningAgent:
    """
    Tabular Q-Learning Agent

    The agent maintains a Q-table:
    - Keys: States (position, battery, time, traffic)
    - Values: Q-values for each action
    """

    def __init__(self, action_space=5, learning_rate=0.1, discount_factor=0.95):
        self.q_table = {}  # Main learning data structure
        self.alpha = learning_rate
        self.gamma = discount_factor
        self.epsilon = 1.0  # Exploration rate
        self.epsilon_decay = 0.995
        self.epsilon_min = 0.01

    def choose_action(self, state, training=True):
        """
        ε-greedy action selection

        With probability ε: random action (explore)
        Otherwise: best known action (exploit)
        """
        # Initialize state if new
        if state not in self.q_table:
            self.q_table[state] = np.zeros(self.action_space)

        if training and np.random.random() < self.epsilon:
            # Exploration: try random action
            return np.random.randint(0, self.action_space)
        else:
            # Exploitation: use best known action
            return np.argmax(self.q_table[state])

    def learn(self, state, action, reward, next_state, done):
        """
        Update Q-value using Bellman equation

        Q(s,a) ← Q(s,a) + α[r + γ·max(Q(s',a')) - Q(s,a)]
        """
        # Initialize states if new
        if state not in self.q_table:
            self.q_table[state] = np.zeros(self.action_space)
        if next_state not in self.q_table:
            self.q_table[next_state] = np.zeros(self.action_space)

        # Current Q-value
        current_q = self.q_table[state][action]

        # Best future Q-value (0 if terminal)
```

```
        max_next_q = 0 if done else np.max(self.q_table[next_state])

        # TD target
        target = reward + self.gamma * max_next_q

        # Update Q-value
        self.q_table[state][action] = current_q + self.alpha * (target -
current_q)

    def decay_exploration(self):
        """Reduce exploration rate over time"""
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)


def train_q_learning_agent(env, agent, episodes=500):
    """
    Training loop
    """
    history = {'rewards': [], 'successes': [], 'lengths': []}

    for episode in range(episodes):
        state = env.reset()
        total_reward = 0
        done = False
        steps = 0

        while not done:
            # Choose action
            action = agent.choose_action(state, training=True)

            # Take action
            next_state, reward, done, truncated, info = env.step(action)

            # Learn from experience
            agent.learn(state, action, reward, next_state, done or truncated)

            # Update
            state = next_state
            total_reward += reward
            steps += 1

            if truncated:
                done = True

        # Decay exploration
        agent.decay_exploration()

        # Record history
        history['rewards'].append(total_reward)
        history['successes'].append(info.get('success', False))
        history['lengths'].append(steps)

        # Print progress
        if (episode + 1) % 50 == 0:
```

```
            avg_reward = np.mean(history['rewards'][-50:])
            success_rate = np.mean(history['successes'][-50:])
            print(f"Episode {episode+1}: Avg Reward = {avg_reward:.1f}, "
                  f"Success = {success_rate*100:.0f}%")

    return history
```

5. environment.py - The Simulator

```
# environment.py - RL environment for EV routing

class EVRoutingEnvironment(gym.Env):
    """
    Gymnasium-compatible environment for EV routing

    The environment simulates an EV navigating a road network
    while managing battery and avoiding traffic.
    """

    def __init__(self, grid_size=10, max_battery=100):
        super().__init__()

        # Create road network
        self.road_graph = RoadGraph(grid_size=grid_size)

        # Define spaces
        self.action_space = spaces.Discrete(5)  # 4 directions + charge
        self.observation_space = spaces.Box(
            low=0, high=1, shape=(8,), dtype=np.float32
        )

        # Episode tracking
        self.ev_state = None
        self.destination = None
        self.current_step = 0

    def reset(self, seed=None):
        """Reset environment for new episode"""
        super().reset(seed=seed)

        # Random source and destination
        self.source = np.random.randint(0, self.road_graph.num_nodes)
        self.destination = np.random.randint(0, self.road_graph.num_nodes)
        while self.destination == self.source:
            self.destination = np.random.randint(0, self.road_graph.num_nodes)

        # Initialize EV state
        self.ev_state = EVState(
            battery_soc=np.random.uniform(50, 100),  # Random starting battery
            current_node=self.source,
            time_minutes=np.random.randint(0, 1440)  # Random time of day
```

```
        )

        self.current_step = 0
        self.route_history = [self.source]

        return self._get_observation(), {}

    def step(self, action):
        """Execute action and return results"""
        self.current_step += 1
        reward = -0.1  # Base step penalty
        terminated = False
        truncated = False
        info = {}

        # Get valid actions
        neighbors = self.road_graph.get_neighbors(self.ev_state.current_node)

        if action == 4:  # Charging action
            if self.ev_state.current_node in self.road_graph.charging_stations:
                # Charge the battery
                old_soc = self.ev_state.battery_soc
                self.ev_state.battery_soc = min(100, self.ev_state.battery_soc +
20)
                reward += (self.ev_state.battery_soc - old_soc) * 0.1
                info['action_type'] = 'charge'
            else:
                reward -= 1  # Penalty for invalid charge attempt
                info['action_type'] = 'invalid_charge'

        elif action < len(neighbors):
            # Move to neighbor
            next_node = neighbors[action]
            edge = (self.ev_state.current_node, next_node)
            cost = self.road_graph.calculate_edge_cost(edge, self.ev_state)

            if cost['feasible']:
                # Update EV state
                self.ev_state.current_node = next_node
                self.ev_state.battery_soc -= (cost['energy_kwh'] /
                    self.ev_state.battery_capacity_kwh) * 100
                self.ev_state.time_minutes += int(cost['time_minutes'])
                self.ev_state.total_distance_km += cost['distance_km']
                self.ev_state.total_energy_kwh += cost['energy_kwh']

                self.route_history.append(next_node)

                # Calculate reward
                reward -= cost['energy_kwh']  # Energy penalty

                # Progress reward
                old_dist = self._distance_to_dest(self.ev_state.current_node)
                # ... (already moved, so this is new position)
```

```
                # Check if reached destination
                if next_node == self.destination:
                    reward += 100
                    terminated = True
                    info['success'] = True

                info['action_type'] = 'move'
            else:
                # Not enough battery
                reward -= 10
                info['action_type'] = 'insufficient_battery'
        else:
            # Invalid action (no such neighbor)
            reward -= 1
            info['action_type'] = 'invalid_move'

        # Check battery empty
        if self.ev_state.battery_soc <= 0:
            reward -= 100
            terminated = True
            info['success'] = False
            info['failure_reason'] = 'battery_empty'

        # Check max steps
        if self.current_step >= 200:
            truncated = True
            info['success'] = False

        return self._get_observation(), reward, terminated, truncated, info

    def _get_observation(self):
        """Get current observation vector"""
        # ... (normalized 8-dimensional vector)
        return np.array([
            node_x, node_y, battery_soc, dist_to_dest,
            time_of_day, traffic_level, is_at_charging, can_reach
        ], dtype=np.float32)
```

🎯 Chapter 10: Practical Exercises

Exercise 1: Understanding the Road Graph

Task: Create a simple 3×3 road graph and find paths.

```
# exercise1_road_graph.py

import networkx as nx
import numpy as np
```

```
# Create a simple 3x3 grid graph
G = nx.grid_2d_graph(3, 3)

# Convert to our format (integer nodes)
mapping = {(i, j): i * 3 + j for i in range(3) for j in range(3)}
G = nx.relabel_nodes(G, mapping)

# Add edge weights (distances)
for u, v in G.edges():
    G[u][v]['distance'] = 1.0  # All edges have distance 1

# Visualize the graph
print("Nodes:", list(G.nodes()))
print("Edges:", list(G.edges()))

# Question 1: How many paths exist from node 0 to node 8?
# Hint: Use nx.all_simple_paths(G, 0, 8)

# Question 2: What is the shortest path?
# Hint: Use nx.shortest_path(G, 0, 8)

# Question 3: If edge (4, 5) has distance 5 instead of 1,
# does the shortest path change?
```

Expected Output:

```
Nodes: [0, 1, 2, 3, 4, 5, 6, 7, 8]
Edges: [(0, 1), (0, 3), (1, 2), (1, 4), (2, 5), (3, 4), (3, 6), (4, 5), (4, 7),
(5, 8), (6, 7), (7, 8)]

All paths from 0 to 8: 12 paths
Shortest path: [0, 1, 4, 5, 8] or [0, 3, 4, 5, 8] (both length 4)
```

Exercise 2: Q-Learning Basics

Task: Implement Q-Learning for a simple grid world.

```
# exercise2_qlearning.py

import numpy as np

# Simple 4x4 grid world
# Goal is at position (3, 3), start at (0, 0)
# Actions: 0=up, 1=right, 2=down, 3=left

class SimpleGridWorld:
    def __init__(self):
        self.state = (0, 0)
        self.goal = (3, 3)
```

```
    def reset(self):
        self.state = (0, 0)
        return self.state

    def step(self, action):
        x, y = self.state

        # Move
        if action == 0 and y < 3:    # Up
            y += 1
        elif action == 1 and x < 3:  # Right
            x += 1
        elif action == 2 and y > 0:  # Down
            y -= 1
        elif action == 3 and x > 0:  # Left
            x -= 1

        self.state = (x, y)

        # Reward
        if self.state == self.goal:
            return self.state, 100, True
        else:
            return self.state, -1, False

# TODO: Implement Q-Learning
# 1. Create Q-table (dictionary)
# 2. Implement epsilon-greedy action selection
# 3. Implement Q-value update
# 4. Train for 1000 episodes
# 5. Print the learned policy

# Your code here:
```

Exercise 3: Understanding GANs

Task: Trace through a GAN training step.

```
Given:
- Real traffic pattern: [0.2, 0.5, 1.5, 1.2, 0.3]  (5 hours)
- Generator output: [0.1, 0.4, 1.8, 1.0, 0.5]
- Discriminator output for real: 0.8 (80% confident it's real)
- Discriminator output for fake: 0.3 (30% confident it's real)

Questions:

1. What is the discriminator loss?
   D_loss = -[log(D(real)) + log(1 - D(fake))]
   D_loss = -[log(0.8) + log(1 - 0.3)]
   D_loss = -[log(0.8) + log(0.7)]
```

```
   D_loss = -[-0.223 + (-0.357)]
   D_loss = 0.58

2. What is the generator loss?
   G_loss = -log(D(fake))
   G_loss = -log(0.3)
   G_loss = 1.20

3. Is the discriminator doing well? (Answer: Yes, it correctly identifies real and
fake)

4. What should happen to the generator? (Answer: It should improve to increase
D(fake))
```

Exercise 4: Build a Simple Route Generator

Task: Create a function that generates routes using different strategies.

```python
# exercise4_route_generator.py

def generate_routes(graph, source, destination, ev_battery):
    """
    Generate candidate routes using multiple strategies.

    Args:
        graph: NetworkX graph
        source: Starting node
        destination: Goal node
        ev_battery: Current battery percentage

    Returns:
        List of (path, energy, time) tuples
    """
    routes = []

    # Strategy 1: Shortest path
    # TODO: Use Dijkstra's algorithm

    # Strategy 2: K-shortest paths (k=3)
    # TODO: Find alternative routes

    # Strategy 3: If battery < 50%, route via charging station
    # TODO: Find path through nearest charging station

    return routes

# Test your implementation
# graph = create_test_graph()
# routes = generate_routes(graph, 0, 99, 40)
# for path, energy, time in routes:
#     print(f"Path: {path}, Energy: {energy:.2f} kWh, Time: {time:.1f} min")
```

Exercise 5: Run the Full System

Task: Train and evaluate the complete system.

```
# Step 1: Navigate to project
cd c:\Users\thavv\OneDrive\Desktop\project_1\EV_Routing\src

# Step 2: Run training (small scale for testing)
python main.py --mode train --grid-size 5 --gan-epochs 10 --episodes 100

# Step 3: Evaluate
python main.py --mode evaluate

# Step 4: Run demo
python main.py --mode demo
```

Questions to answer after running:

1. How many states did the Q-Learning agent learn?
2. What is the success rate of the trained agent?
3. How does the GNN-generated route compare to the shortest path?

Glossary

Term	Definition
Action	A decision the agent can make (e.g., move north, charge)
Adjacency Matrix	A matrix showing which nodes are connected
Backpropagation	Algorithm for calculating gradients in neural networks
Bellman Equation	Recursive formula for optimal value functions
Discriminator	GAN component that distinguishes real from fake data
Edge	A connection between two nodes in a graph
Epoch	One complete pass through the training data
Exploration	Trying random actions to discover new strategies
Exploitation	Using the best known strategy
GAN	Generative Adversarial Network - two networks competing
Generator	GAN component that creates fake data
GNN	Graph Neural Network - neural network for graph data

Term	Definition
Gradient	Direction and rate of steepest increase of a function
Graph	Mathematical structure of nodes and edges
Heuristic	An educated guess used to guide search
Learning Rate (α)	How much new information overrides old information
Loss Function	Measures how wrong the model's predictions are
Node	A point in a graph (represents intersections)
Policy	Strategy for choosing actions
Q-Learning	RL algorithm that learns state-action values
Q-Table	Lookup table storing Q-values for state-action pairs
Q-Value	Expected future reward for taking action in state
Reward	Feedback signal indicating how good an action was
RL (Reinforcement Learning)	Learning through trial and error
SoC (State of Charge)	Battery level as a percentage
State	Current situation the agent observes
Weight	Learnable parameter in a neural network

🕮 Further Reading

Books

1. **"Reinforcement Learning: An Introduction"** - Sutton & Barto
    - The bible of RL, available free online
2. **"Deep Learning"** - Goodfellow, Bengio, Courville
    - Comprehensive deep learning textbook
3. **"Graph Neural Networks: Foundations, Frontiers, and Applications"**
    - Wu et al., comprehensive GNN guide

Online Courses

1. **Coursera: Machine Learning Specialization** (Andrew Ng)
    - Great introduction to ML fundamentals
2. **DeepMind: Introduction to Reinforcement Learning**

- Free YouTube series on RL

3. **Stanford CS224W: Machine Learning with Graphs**
    - GNN fundamentals (free videos)

Papers (Simplified Explanations)

1. **"Generative Adversarial Networks"** (Goodfellow et al., 2014)
    - The original GAN paper
2. **"Graph Attention Networks"** (Veličković et al., 2018)
    - Introduced attention mechanism for graphs
3. **"Playing Atari with Deep Reinforcement Learning"** (Mnih et al., 2013)
    - Deep Q-Learning breakthrough

Code Resources

1. **TensorFlow Tutorials**: https://www.tensorflow.org/tutorials
2. **NetworkX Documentation**: https://networkx.org/
3. **Gymnasium Documentation**: https://gymnasium.farama.org/

🎓 Final Thoughts

Congratulations on completing this learning book! You now understand:

☑ **Graph Theory**: How road networks are represented mathematically

☑ **EV Fundamentals**: Battery management, energy consumption, charging

☑ **Pathfinding**: Dijkstra, A*, and energy-optimal routing

☑ **Reinforcement Learning**: Q-Learning for decision making

☑ **Neural Networks**: How AI learns from data

☑ **GANs**: Generating realistic traffic patterns

☑ **GNNs**: Understanding network structures

☑ **Complete System**: How all components work together

Key Takeaways

1. **Complex problems need multiple approaches**: We combine classical algorithms (Dijkstra), machine learning (GAN), and reinforcement learning (Q-Learning) for best results.
2. **Data representation matters**: Converting road networks to graphs enables powerful algorithms.

3. **Learning takes time**: Both Q-Learning and GANs need many iterations to perform well.
4. **Real-world applications**: This technology is used in Google Maps, Tesla navigation, and ride-sharing apps.

Next Steps

1. **Run the code**: Practice with the exercises
2. **Modify parameters**: Try different grid sizes, learning rates
3. **Add features**: Weather conditions, real traffic data
4. **Scale up**: Integrate with real map data (OpenStreetMap)

Happy Learning! 🚗⚡🤖

This book was created to accompany the EV Routing System project.